Reto técnico de automatización

Nombre: Maritza Nataly Rosero Velasco

1. Pasos realizados

Preparación del proyecto. Creé el proyecto en IntelliJ IDEA con Java 17 y Gradle. Configuré Serenity BDD con el patrón Screenplay para organizar las acciones del usuario, Selenium WebDriver para interactuar con Chrome, JUnit 5 para ejecutar las verificaciones y Apache PDFBox para leer el documento PDF. Esta separación permite mantener los localizadores de la página independientes del flujo de la prueba.

Navegación en Banistmo. La prueba abre la sección de empresas del sitio de Banistmo, acepta el aviso de privacidad, selecciona «Productos & servicios» y luego «Cuentas». Después localiza «Cuenta de Ahorro Comercial» y accede a su información. Utilicé acciones de desplazamiento (Scroll) antes de algunos clics porque los elementos se encuentran más abajo en la página.

Acceso al documento. Dentro del producto, la automatización se desplaza a «Documentos», selecciona esa sección y abre «Contrato único de Productos y Servicios Bancarios». El archivo se presenta en una nueva pestaña, por lo que la prueba identifica las ventanas abiertas y cambia el foco a la pestaña del PDF.

Validación del PDF. Primero compruebo que se abrió una segunda pestaña y que su dirección contiene «.pdf». Luego leo el archivo con PDFBox, extraigo su texto y verifico que no esté vacío, que el documento tenga páginas y que el contenido incluya «Contrato único de Productos y Servicios Bancarios». Para comparar el título, normalizo los espacios y convierto el texto a minúsculas; así, los saltos de línea o el formato del PDF no afectan la comprobación.

Resultado obtenido. La última ejecución finalizó con BUILD SUCCESSFUL. El archivo consultado correspondió a la revisión de mayo de 2026 y contenía 37 páginas. La comprobación confirma el título y la lectura del documento.

2. Repositorio del código

El proyecto se encuentra publicado en un repositorio público de GitHub. Incluye el código de la prueba, los localizadores, la configuración de Serenity y los archivos necesarios para ejecutar el proyecto con Gradle.

Repositorio: https://github.com/mnrosero/AutomatizacionBanistmo

3. Comandos Git utilizados

Durante la preparación y publicación del proyecto utilicé los siguientes comandos, en el orden correspondiente:

`git --version`
Comprobar que Git estuviera instalado.

`git init`
Inicializar el repositorio local.

`git status`
Revisar los archivos pendientes y su estado; se ejecutó varias veces.

`git add .`

Preparar los archivos para el commit.

`git config --global user.name "Nataly Rosero"`

Configurar el nombre

`git config --global user.email "mariu_0.5@hotmail.com"`

Configurar el correo.

`git config --global --list`

Comprobar la configuración de Git.

`git commit -m "Automatizacion Banistmo con Java Gradle Screenplay y validacion PDF"`

Guardar la primera versión del proyecto. El primer intento requirió configurar la identidad de Git.

`git remote add origin https://github.com/mnrosero/AutomatizacionBanistmo.git`

Vincular el repositorio local con GitHub.

`git remote -v`

Verificar la dirección del repositorio remoto.

`git push -u origin master`

Publicar la rama master en GitHub.

Como comprobación final, revisé el estado del repositorio con git status y confirmé que no había cambios pendientes.